# Reporte de Agentic Loop

## 0. SKILLS CONFIGURADOS

arquitecto !’ DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !’ MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !’ FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !’ test (GLX-QA-VALIDACION-V1)

## 1. ANÁLISIS DE ARQUITECTURA

## Micro-ADR — XP-14: Página Web Metodologías TI

### 1. CONTEXTO
Se requiere una página web estática de contenido editorial sobre metodologías de informática y TI. El stack está explícitamente definido por el cliente: HTML5, CSS y JS puro. No existe backend, base de datos, autenticación de usuarios ni lógica de negocio transaccional. El entorno de ejecución final es un servidor HTTP/HTTPS (el criterio de seguridad exige HTTPS y Content-Security-Policy en cabeceras), lo que técnicamente permite módulos ES6. Sin embargo, dado que es un sitio estático de contenido puro sin bundler (Webpack/Vite), se optará por un único archivo JS consolidado para máxima portabilidad y compatibilidad sin toolchain de build.

### 2. DECISIÓN
**Arquitectura: Monolito Estático de Archivo Único (Single-File Static Site)**
- 1 archivo HTML principal con 3 secciones SPA-like (Home, Historias, Noticias) navegadas por anclas y JS de routing mínimo.
- 1 archivo CSS consolidado con variables custom properties para theming responsivo.
- 1 archivo JS plano (sin módulos ES6 nativos) que agrupa: router de secciones, FAQ accordion, cookie consent, sanitización XSS con DOMPurify (cargado desde CDN con SRI hash), y stub de Google Analytics.
- Assets de imágenes: URLs de placeholder (placehold.co) hasta integración final.

### 3. JUSTIFICACIÓN BASADA EN YAGNI / KISS / BORING TECH
- **NO se usa React/Vue/Angular**: YAGNI. El contenido es estático y editorial. Un framework JS añadiría 40-200KB de bundle, tiempo de hidratación y complejidad de build sin ningún beneficio funcional para este caso de uso.
- **NO se usa un SSG (Gatsby, Next, Hugo)**: YAGNI. No hay CMS, no hay datos dinámicos, no hay rutas generadas programáticamente. Añadiría un pipeline de build innecesario.
- **NO se divide en múltiples archivos JS con import/export**: El entorno carece de bundler definido. Usar type='module' sin bundler rompe la portabilidad en servidores simples (Apache/Nginx sin configuración especial) y complica el despliegue para el equipo. Un único main.js es la solución más robusta.
- **NO se usa MongoDB, Redis ni ninguna base de datos**: El contenido es hardcodeado en el HTML/JS. No hay persistencia requerida.
- **DOMPurify vía CDN con Subresource Integrity (SRI)**: Solución madura, probada, zero-config para sanitización XSS. Alternativa de implementar sanitización manual sería insegura y violaría 'Choose Boring Technology'.
- **Google Analytics via gtag.js**: Solución estándar de industria para el criterio de análisis de tráfico. No se construye sistema propio.

### 4. RIESGOS ASUMIDOS POR SIMPLIFICAR
- **Riesgo 1 (BAJO)**: El contenido hardcodeado en HTML/JS dificulta actualizaciones editoriales frecuentes. Mitigación: estructura de datos JS clara (arrays de objetos) que actúa como pseudo-CMS. Si el volumen de contenido crece >50 artículos, se recomienda migrar a un SSG en una iteración futura.
- **Riesgo 2 (BAJO)**: Content-Security-Policy estricta puede bloquear CDNs si no se configuran correctamente los hashes SRI y dominios permitidos. Mitigación: el meta CSP en HTML cubre el entorno de desarrollo; en producción debe trasladarse a cabeceras HTTP del servidor (Nginx/Apache config).
- **Riesgo 3 (MUY BAJO)**: Sin backend, el aviso de cookies no puede persistir el consentimiento en servidor. Mitigación: localStorage para persistir la preferencia del usuario en el cliente, que es el estándar para sitios estáticos.

- **Riesgo 4 (INFORMATIVO)**: HTTPS no puede implementarse a nivel de código estático; es responsabilidad de la capa de infraestructura (Nginx + Let's Encrypt o CDN como Cloudflare). Se documenta en el contrato de interfaces.

## 2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO (integration-fix iter 1) | Iteraciones: 1
- assets/js/main.js | Estado: APROBADO (integration-fix iter 1) | Iteraciones: 4
- assets/css/styles.css | Estado: APROBADO (integration-fix iter 1) | Iteraciones: 1
- README.md | Estado: SKIPPED (doc) | Iteraciones: 0

## 3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 169,711
Tokens de salida (output): 54,979
Total tokens: 224,690
Modelo utilizado: claude-sonnet-4-6

Ø=Üh  Ø=Ü» ~23.8 horas-hombre ahorradas
"H squad de 4 personas × 3 días

Desglose por Agente:

DISEÑO-ARQUITECTURA: !“910 input | !‘1,363 output | 1 llamadas | ~1.5h ahorradas
MICROPLANIFICACION-DISEÑO: !“12,121 input | !‘9,500 output | 4 llamadas | ~5.6h ahorradas
FRONTEND-DESARROLLO: !“80,112 input | !‘32,039 output | 8 llamadas | ~9.4h ahorradas
test: !“76,568 input | !‘12,077 output | 8 llamadas | ~7.3h ahorradas